Введение в Go и настройка окружения для разработки

Лекция 01: Введение в Go

Материалы (контент) могут использоваться исключительно в учебных и информационных целях без цели извлечения прибыли с обязательным указанием следующей информации:
© ООО «КНС ГРУПП» (YADRO), 2022-2023, все права защищены.
Авторы – Александрова О. В., Андрианов А.А., Денисенко Б. А., Дутова А. А., Ефимов М. А., Клочкова Л. И., Королев Д. О., Котлярова Л. П., Никифоров И. В., Ромашов В. А., Тянутов М. В.

История

Go представляет собой компилируемый статически типизированный язык программирования от компании Google. Полное название Golang (Google language). Язык был впервые анонсирован 10 ноября 2009 года.

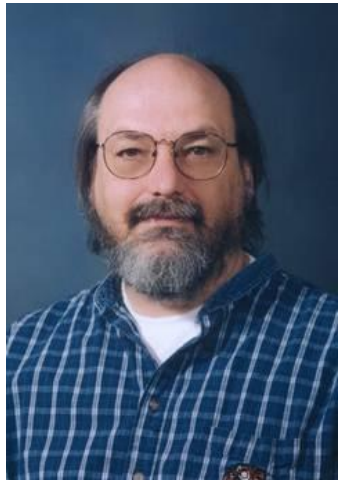
Область применения:

- Веб-сервисы;
- Многопоточные приложения.

Создатели: Роб Пайк

pt.wikipedia.org, GNU Free Documentation License

Кен Томпсон

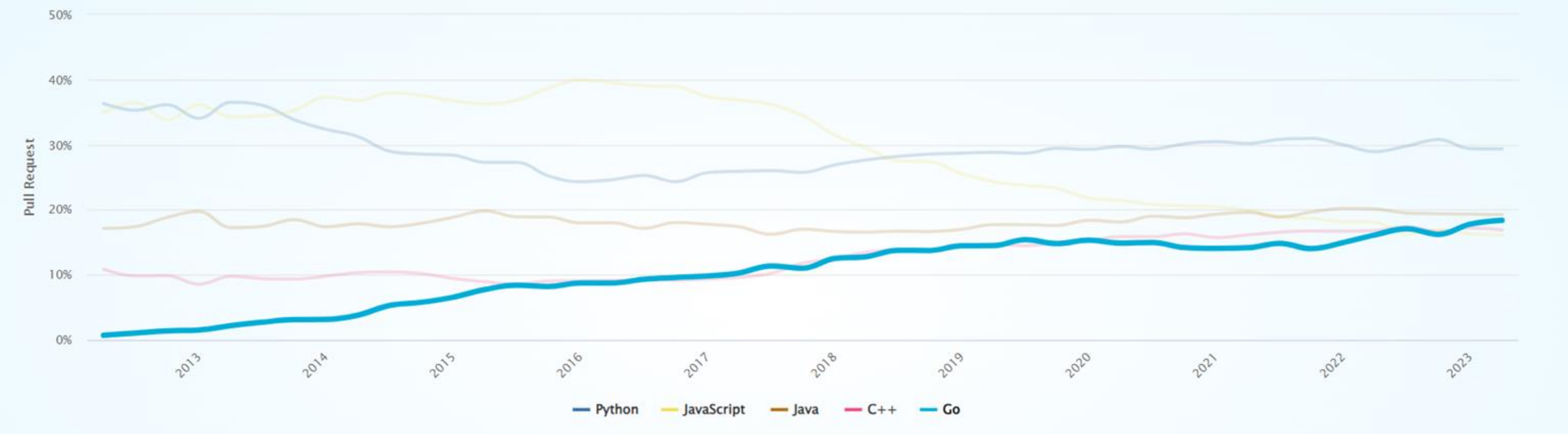


pt.wikipedia.org, Creative Commons Attribution 2.0 Generic

C

Актуальность

Go – молодой язык, который быстро набирает популярность. По статистике количества Pull Request на GitHub он уже занимает 3-е место, уступая только Python и Java, а рост популярности этого языка – самый высокий из всей пятерки лидеров (за последний год +1.5% от всех Pull Requests).



Тенденция популярности языков программирования, основанная на количестве Pull Requests на GitHub

Источник: https://madnight.github.io/githut/#/pull_requests/2023/2

Происхождение

Go унаследовал от:

C – синтаксис выражений, базовые типы данных, понятие указателей и направленность на получение при компиляции эффективного машинного кода.

Pascal, Oberon – модули, пакеты.

Squeak, Alef – параллелизм.

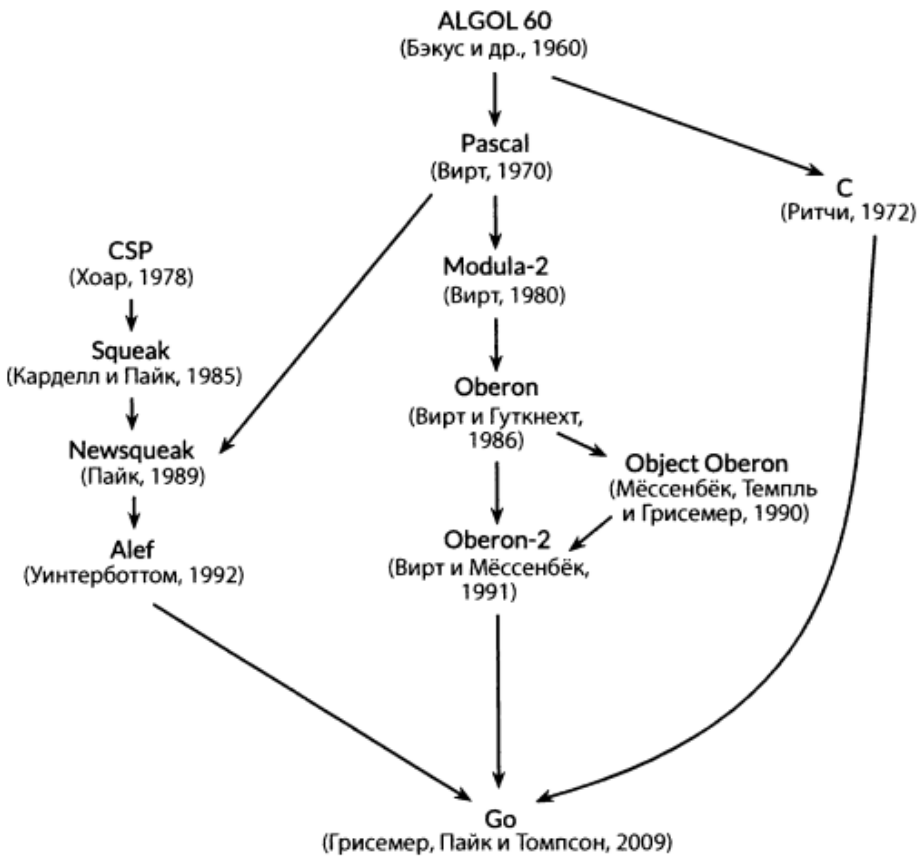


Схема происхождения языка Go

Источник: The Go Programming Language, Donovan Kernigan

Отличия от других языков

Основные особенности языка Go:

- Компилируемый;
- Статически типизированный;
- Присутствует сборщик мусора, который автоматически очищает память;
- Поддержка работы с сетевыми протоколами;
- Поддержка многопоточности и параллельного программирования;
- Отсутствие оператора try-catch.

Отличия от других языков

Некоторые тонкости языка Go:

- Компиляция будет провалена, если переменная была объявлена, но не использована;
- В Go открывающая фигурная скобка должна находиться на одной строке с конструкцией;
- Функции и поля, названия которых начинаются с маленького регистра, являются видимыми только в пределах одного пакета.

Отличия от других языков

Объявление переменных:

```
var x int
var a, b int
var (
  x int
  name string
)

var x int = 42
var x = 42

x := 42
x, y := 1, 2

_ = 42
```

Важный момент: краткое объявление переменной не обязательно объявляет все переменные в своей левой части. Если некоторые из них уже были объявлены, то для этих переменных краткие объявления действуют как присваивания.

Однако краткое объявление переменной должно объявлять по крайней мере одну новую переменную, так что приведенный ниже код не компилируется:

```
f, err := os.Open(infile)
// ...
f, err := os.Create(outfile) // Ошибка: нет новой переменной
```

Отличия от других языков

Оператор **switch case** не требует **break** после каждого **case**. Для того чтобы активировать следующий **case** используется оператор **fallthrough**.

```go
package main

import "fmt"

func test(value int) {
    switch value {
    case 1:
        fmt.Println("Один")
        fallthrough
    case 0:
        fmt.Println("Ноль")
    }
}

func main() {
    test(0)
    test(1)
}
```

Вывод:

Ноль

Один

Ноль

```go
package main

import "fmt"

func test(value int) {
    switch value {
    case 1:
        fmt.Println("Один")
    case 0:
        fmt.Println("Ноль")
    }
}

func main() {
    test(0)
    test(1)
}
```

Вывод:

Ноль

Один

Отличия от других языков

В языке нету цикла **while**, вместо него можно использовать **for** с условием на переменную.

```go
package main

import "fmt"

func main() {
    sum := 1
    for sum < 1000 {
        sum += sum
    }
    fmt.Println(sum)
}
```

Вывод:

1024

Отличия от других языков

Выражение **range** возвращает 2 значения – индекс и значение.

```go
package main

import "fmt"

func main() {
    x := []string{"a","b","c"}

    for v := range x {
        fmt.Println(v)
    }
}
```

Вывод:

0, 1, 2

```go
package main

import "fmt"

func main() {
    x := []string{"a","b","c"}

    for _, v := range x {
        fmt.Println(v)
    }
}
```

Вывод:

a, b, c

Отличия от других языков

Строка в Golang – это обычный массив байт.

```
type _string struct {
    elements *byte // underlying bytes
    len      int   // number of bytes
}
```

```
s := "Japan 日本"
for i := 0; i < len(s); i++ {
    fmt.Printf("%q ", s[i])
}
```

Вывод: 'J' 'a' 'p' 'a' 'n' ' ' 'æ' '\u0097' '¥' 'æ' '\u009c' '¬'

Отличия от других языков

Нет ООП в привычном понимании этого термина.

```
type Parent struct{}

func (c *Parent) Print() {
    fmt.Println("parent")
}

type Child struct {
    Parent
}

func (p *Child) Print() {
    fmt.Println("child")
}

func main() {
    var x Child

    x.Print()
}
```

→ **Вывод:** child

Отличия от других языков

Начиная с Go 1.18 поддерживаются шаблоны

```go
package main

import "fmt"

func Swap[T any](a, b T) (T, T) {
  return b, a
}

func main() {
  a, b := 3, 4
  a, b = Swap(a, b)
  fmt.Println(a, b)
  x, y := "hello", "world"
  x, y = Swap(x, y)
  fmt.Println(x, y)
}
```

Вывод: world hello

Отличия от других языков

В Go имеется поддержка анонимных функций

```go
package main

import (
  "fmt"
)

func processString(str string, fn
func(string) string) string {
  return fn(str)
}

func main() {
  inputString := "Hello, World!"

  result :=
processString(inputString, func(s
string) string {
     return "Processed: " + s
  })

  fmt.Println(result)
}
```

Вывод: Hello, World!

Область применения

GO - разработан в Google для нужд компании, поэтому этот язык лучше всего подходит для работы с веб-сервисами.

Основные юзкейсы Go:

- Облачные приложения;
- Веб-разработка;
- консольные утилиты;
- Реализации масштабируемых баз данных;
- Сетевое программное обеспечение.

Для создания горутины (аналога треда), достаточно написать "go" перед вызовом функции, которую мы хотим запустить в отдельной горутине. В отличие от тредов в других языках программирования, горутины занимают очень мало памяти - примерно в 1000 раз меньше.

```go
package main

import (
  "fmt"
  "time"
)

func add(a int, b int) {
  time.Sleep(1 * time.Second)
  fmt.Println("Sum = ", a+b)
}

func sub(a int, b int) {
  fmt.Println("Sub = ", a-b)
}

func main() {
  go add(5, 4)
  go sub(5, 4)
  time.Sleep(2 * time.Second)
}
```

Вывод:

Sub = 1

Sum = 9

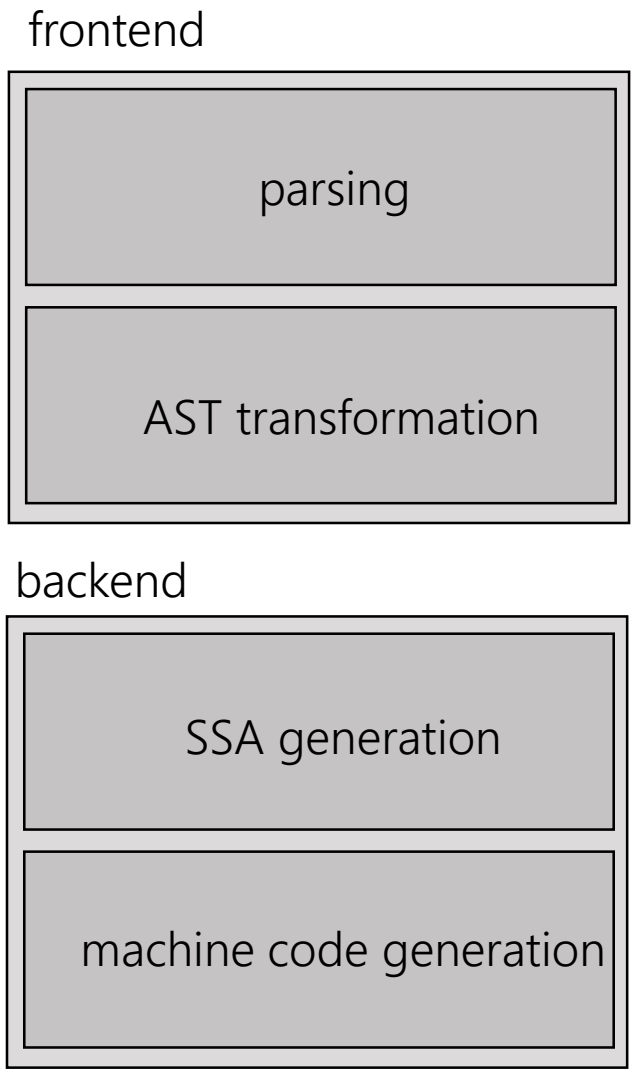
Компиляция

Компиляция в Go состоит из четырёх фаз, которые можно объединить в два этапа:

- На первом выполняется анализ исходного кода и по мере синтаксического разбора создаётся абстрактная синтаксическая структура исходного кода.
- На втором этапе вместе с многочисленными оптимизациями происходит трансформация представления исходного кода в машинный код.

Компилятор Go поддерживает кроссплатформенность. С помощью переменных среды GOARCH и GOOS можно установить целевую архитектуру и ОС

```
set GOARCH=amd64
set GOOS=linux
go build
```



Установка GO

Для работы языка Go достаточно установить компилятор с официального сайта (https://go.dev/dl/). Это обязательный пункт для работы языка Go независимо от выбранной IDE.

В качестве IDE можно использовать Goland от JetBrains, либо LiteIDE с открытым исходным кодом. Также можно использовать плагин языка для других IDE, например Visual Studio или Eclipse.

После установки компилятора и IDE/плагина можно сразу приступать к работе.

Структура проекта в GO

Проект имеет в основе следующую структуру:

- /cmd — папка в которой хранится точка входа в приложение в виде функции main();
- /pkg — папка в которой хранятся пакеты необходимые для работы приложения и которые мы хотим использовать в других проектах;
- /internal — папка в которой хранятся пакеты которые мы хотим использовать только в этом проекте. В языке Go такая папка просто игнорируется при поиске пакетов вне этого проекта.

Подробнее узнать о структуре можно по ссылке https://github.com/golang-standards/project-layout/blob/master/README_ru.md

Пример структуры проекта

Работа с зависимостями

go.mod

Файл **go.mod** является описанием модуля Go и содержит информацию о его зависимостях и версиях.

В go.mod вы указываете:

- Имя вашего модуля
- Зависимости вашего модуля
- Зависимости, которые необходимо исключить
- Зависимости, которые можно заменить

Go использует информацию из go.mod, чтобы определить, какие версии зависимостей следует скачать и использовать для вашего проекта

go.sum

Файл go.sum содержит хэши контрольных сумм зависимостей, используемых вашим модулем. Эти контрольные суммы обеспечивают целостность скачанных пакетов и предотвращают атаки на безопасность.

Работа с зависимостями

Работа с зависимости в Go осуществляется с помощью системы модулей.

Для работы с несколькими пакетами необходимо инициализировать go.mod с помощью команды **go mod init *ProjectName***. После этого можно без проблем импортировать пакеты из этого проекта.

Сторонние библиотеки можно легко установить с помощью команды **go get *название***. (библиотеки можно найти на официальном сайте pkg.go: https://pkg.go.dev/).

Библиотеке можно присвоить псевдоним для удобства, он указывается перед названием самой библиотеки при импорте.

```
type Plot in gonum.org/v1/plot
Package plot provides an API for setting up plots
```

Пример названия модуля

```go
import jr "github.com/andygrunwald/go-jira"

func main() {
  tp := jr.BasicAuthTransport{
     Username: "exapmle@gmail.com",
     Password: "pass",
  }
}
```

Пример псевдонима для библиотеки go-jira

Тесты

- Тесты создаются в директории с файлом, который необходимо протестировать;
- Формат названия файла с тестами: "*_test.go";
- Для тестирования в Go есть встроенная библиотека "testing".

Пример тестирования функции сортировки "пузырьком":

```go
func BubbleSort(ar []int) {
  for i := 0; i < len(ar); i++ {
     for j := len(ar) - 1; j > i; j-- {
        if ar[j-1] > ar[j] {
           ar[j-1], ar[j] = ar[j], ar[j-1]
        }
     }
  }
}
```

sort.go

```go
func TestBubbleSort(t *testing.T) {
  ar := []int{5, 35, 12, 24, 21, 98, 6, 10}
  expected := []int{5, 6, 10, 12, 21, 24, 35, 98}

  mySort.BubbleSort(ar)

  for inx, elem := range ar {
     if elem != expected[inx] {
        t.Errorf("Bubble sort is not correct.
Expect %d, got %d", expected, ar)
        break
     }
  }
}
```

sort_test.go

Сборка

go run — используется для запуска программы без сборки;
go build — используется для сборки программы. Программа собирается в исполняемый файл со всеми нужными зависимостями внутри;
go install — сборка и установка в систему. Установка подразумевает под собой перенос исполняемого файла в директорию *$GOPATH/bin*, благодаря чему программу можно запустить из командной строки в любом месте.

Пример команд:

- **go run cmd/main.go** - запускает программу без сборки бинарного файла;
- **go build -o program.exe ./cmd/main.go** - собирает программу в бинарный файл с названием program.exe.

Команды и переменные среды

Переменные среды

GOROOT — здесь установлено само окружение Go. Это местоположение стандартной библиотеки и инструментов компиляции Go.

GOPATH — это место, куда Go будет устанавливать сторонние пакеты.

GOPROXY — прокси-сервер, который кэширует зависимости, чтобы уменьшить нагрузку на официальные репозитории.

Команды

go mod tidy — нужные зависимости загружаются, а ненужные из go.mod удаляются

go mod vendor — создать каталог, который содержит копии всех зависимостей вашего проекта. Это полезно, если вы хотите включить зависимости в репозиторий вашего проекта и изолировать его от глобального пространства модулей.

go get — используется для загрузки и установки сторонних пакетов и зависимостей в вашем проекте.

go install — используется для компиляции и установки бинарных файлов (исполняемых программ) или библиотек в соответствующую директорию, определенную в переменной окружения GOBIN.

Инструкция как создать проект в GO

- Заходим на сайт https://go.dev/dl/
- Скачиваем и устанавливаем компилятор
- Устанавливаем IDE или плагин для IDE
- Создаем новый проект (пустую папку)
- Переходим в директорию проекта
- Пишем go mod init *название папки*
- Создаем структуру проекта из папок cmd и pkg
- В папке cmd создаем файл main.go
- В файле main.go указываем package main и функцию main()
- Для создания новых пакетов в папке pkg создаем новую папку и создаем в ней файл
- В файле указываем package *название папки*
- Чтобы функции были видны в других пакетах, например в main, необходимо, чтобы название функции начиналось с большой буквы.

Полезные ссылки

- Официальный сайт go: https://go.dev/;
- GoLand: https://www.jetbrains.com/go/;
- LiteIDE: https://github.com/visualfc/liteide/blob/master/liteidex/deploy/welcome/en/install.md;
- Go плагин для Visual Studio: https://marketplace.visualstudio.com/items?itemName=golang.go;
- Go плагин для Eclipse: https://goclipse.github.io/.

yadro.com